

11 function to delete a node from BST

```
public static Node delNode(Node root, int key){
```

```
    if (root == null)
        return root;
```

```
    if (key < root.data)
        root.left = delNode(root.left, key);
```

```
    else if (key > root.data)
        root.right = delNode(root.right, key);
```

```
    else {
```

```
        if (root.left == null)
            return root.right;
```

```
        else if (root.right == null)
            return root.left;
```

```
        root.data = minValue(root.right);
```

```
        root.right = delNode(root.right, root.data);
```

```
    }
```

```
    return root;
```

```
}
```

```
static int minValue(Node root) {
```

```
    int minv = root.data;
```

```
    while (root.left != null) {
```

```
        minv = root.left.data;
```

```
        root = root.left;
```

```
    }
```

```
    return minv;
```

```
}
```

जब कभी भी हम Node insert कर रहे होते हैं BST में तब हमें leaf node पर insert होता है या नहीं leaf node बनता है जब हमें height पर ध्यान नहीं दे रहे होते हैं।

// Insert in BST

```
Node insert (Node root, int key) {
```

```
    if (root == null)
        return new Node(key);
```

```
    if (root.data > key)
        root.left = insert (root.left, key);
```

```
    else if (root.data < key)
        root.right = insert (root.right, key);
```

```
    return root;
```

3

Node insert (Node root, int key) {

Node newNode = new Node(key);

Node cur = root;

Node parent = null;

while (cur != null) {

parent = cur;

if (key < cur.key)

cur = cur.left;

else

cur = cur.right;

}

if (parent == null)

^{return}
~~parent~~ = newNode;

else if (key < parent.key)

parent.left = newNode;

else

parent.right = newNode;

return ~~parent~~, root;

3

1) Function to check whether a Binary tree is BST or Not.

```
Boolean isBST(Node root) {
    return isBSTUtil(root, Integer.MIN_VALUE, Integer.MAX_VALUE);
}
```

3

```
Boolean isBSTUtil(Node node, int min, int max) {
```

```
    if (node == null)
        return true;
```

```
    if (node.data < min || node.data > max)
        return false;
```

```
    return (isBSTUtil(node.left, min, node.data - 1) &&
            isBSTUtil(node.right, node.data + 1, max));
}
```

3

Binary Search Tree

	Search	Insert	Delete
Array	$O(n)$	$O(1)$	$O(n)$
Array (Sorted)	$O(\log n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(1) O(n)$	$O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash Table	$O(1)$	$O(1)$	$O(1)$

// function to return the LCA in BT

```
Node lca(Node root, int n1, int n2)
```

```
{
```

```
    if (root == null)
        return null;
```

```
    if (root->data == n1 || root->data == n2)
        return root;
```

```
    Node left = lca(root->left, n1, n2);
```

```
    Node root right = lca(root->right, n1, n2);
```

```
    if (left == null)
        return right;
```

```
    if (right == null)
        return left;
```

```
    return root;
```

(3) 0 3

(1) 0

(3) 0

(3) 0

(3) 0

(3) 0

(3) 0

(1) 0 (3) 0

(3) 0

(3) 0

(3) 0

(3) 0

(3) 0

(3) 0

(3) 0

Ans,

find a pair with the given sum in a BST.

// maximum in a B.T

```
int maximum(Node root)
{
    if (root == null)
        return Integer.MIN_VALUE;
    return Math.max(root.data, Math.max(
        maximum(root.left), maximum(root.right)
    ));
}
```

3

imp Ques:

① Number of nodes in the largest path b/w two leaf nodes.

② minimum time to burn a binary tree

③ AVL tree insertion.

// Vertical Traversal in a BT

```
Static ArrayList<Integer> verticalOrder(Node root) {
```

```
    Queue<Pair> q = new ArrayDeque<>();
```

```
    Map<Integer, ArrayList<Integer>> map =
```

```
    new TreeMap<>();
```

```
    q.add(new Pair(0, root));
```

```
    while(!q.isEmpty()) {
```

```
        Pair cur = q.poll();
```

```
        if (map.containsKey(cur.hd)) {
```

```
            map.get(cur.hd).add(cur.node.data);
```

```
        }
```

```
        else {
```

```
            ArrayList<Integer> temp = new ArrayList<>();
```

```
            temp.add(cur.node.data);
```

```
            map.put(cur.hd, temp);
```

```
        }
```

```
        if (cur.node.left != null)
```

```
            q.add(new Pair(cur.hd - 1, cur.node.left));
```

```
        if (cur.node.right != null)
```

```
            q.add(new Pair(cur.hd + 1, cur.node.right));
```

```
    }
```

```
    ArrayList<Integer> ans = new ArrayList<>();
```

```
    for (Map.Entry<Integer, ArrayList<Integer>> e : map.entrySet())
```

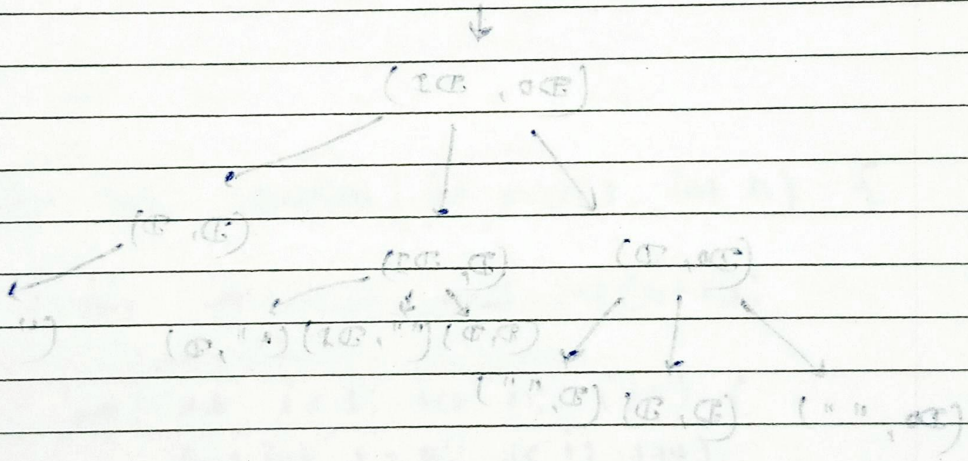
```
        ans.addAll(e.getValue());
```

```
    return ans;
```

```
static class Pair {  
    int hd;  
    Node node;  
    Pair() {  
        hd = 0;  
        node = null;  
    }  
}
```

variations of LCS

- ① LCS space complexity Optimisation.
- 2) minimum Insertion and Deletions to Convert S_1 into S_2
3. Length of Super Sequence of S_1 and S_2
4. Longest repeating Subsequence
5. Longest palindromic Subsequence



8	A	7	8			
2	P	E	S	1	0	
2	P	E	S	1	1	E
P	E	E	S	1	S	A
P	E	E	S	S	E	C
E	E	E	S	E	P	B

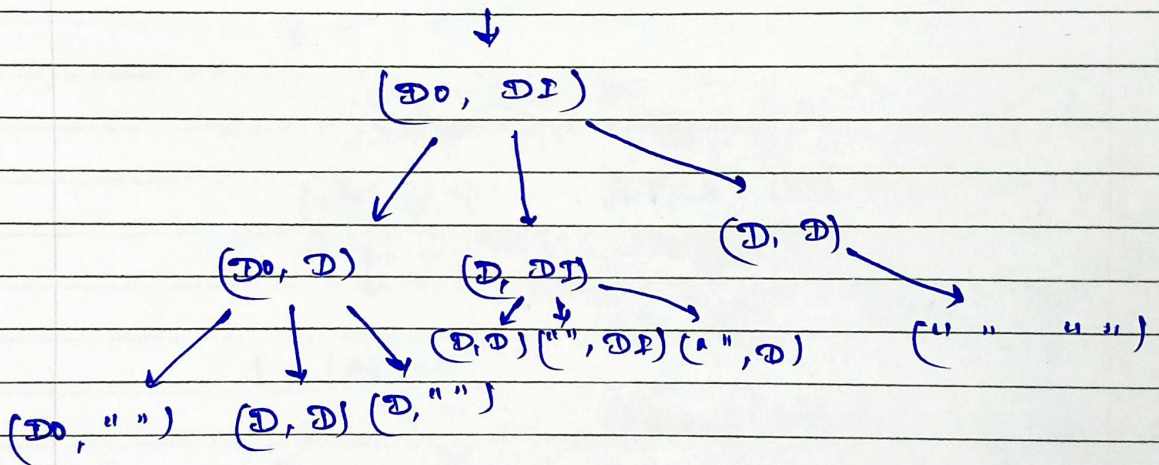
string s1 into s2

Insert

Delete

Replace

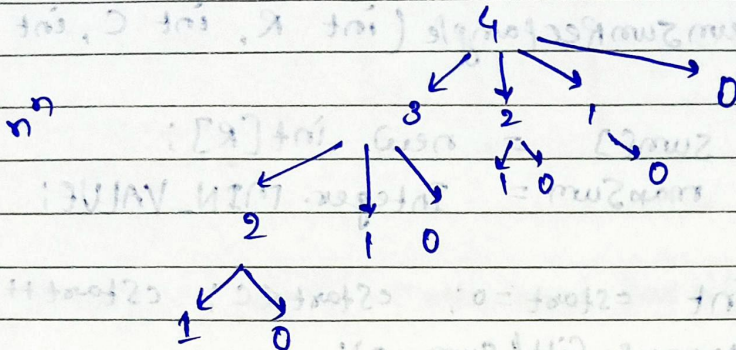
$S_1 = \{D, O, G\}$

$$S_2 = DEh$$


	A	B	C	A	B
E	0	1	2	3	4
A	1	1	2	3	4
C	2	1	2	3	4
B	3	2	2	3	4
	4	3	2	3	3

Cutting a rod

length	1	2	3	4	5	6	7	8
Price	1	5	6	9	11	12	14	16



```
public int cutRod(int price[], int n) {
```

```
    int dp[] = new int[n+1];
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 0; j < i; j++)
```

```
            dp[i] = Math.max(dp[i], price[j] + dp[i-j-1]);
```

```
    }
```

```
    return dp[n];
```

```
}
```

Maximum Sum Rectangle.

Given a 2D matrix M of dimensions $R \times C$.

Find the maximum sum submatrix in it.

```
int maximumSumRectangle(int R, int C, int M[R][C])  
{
```

```
    int sum[] = new int[R];
```

```
    int maxSum = Integer.MIN_VALUE;
```

```
    for (int cstart = 0; cstart < C; cstart++) {  
        Arrays.fill(sum, 0);
```

```
        for (int cend = cstart; cend < C; cend++) {
```

```
            for (int row = 0; row < R; row++)
```

```
                sum[row] += M[row][cend];
```

```
            int curMaxSum = Kadane(sum);
```

```
            maxSum = max(maxSum, curMaxSum);
```

```
        }
```

```
    }
```

```
    return maxSum;
```

```
}
```

Palindromic partitioning
minimum number of partitions to make palindrome

	a_0	b_1	a_2	c_3
0	0	1	0	3
1	1	0	1	2
2			0	1
3				0

```
static int palindromicPartition(String s) {
```

```
    int n = s.length();
    int dp[][] = new int[n][n];
```

```
    for (int gap = 1; gap < n; gap++) {
        for (int row = 0, col = gap; row < n - gap; row++, col++) {
```

```
            if (isPalindrome(s, row, col))
                dp[row][col] = 0;
```

```
            else {
                dp[row][col] = MAX_VALUE;
```

```
                for (int k = row; k < col; k++) {
                    dp[row][col] = min(dp[row][col],
                        1 + dp[row][k] + dp[k+1][col]);
```

```
            }
        }
    }
    return dp[0][n-1];
```

Maximum Meeting in one Room

$S[] = \{1, 4, 12, 15, 7\}$
 $f[] = \{13, 6, 14, 19, 11\}$

```
public static ArrayList<Integer> maxmeetings(int N,  
int []S ) {
```

```
    ArrayList<Integer> ans = new ArrayList<>();  
    int a[][] = new int[N][3];  
    for (int i = 0; i < N; i++) {  
        a[i][0] = i + 1;  
        a[i][1] = S[i];  
        a[i][2] = f[i];
```

```
    }
```

```
    Arrays.sort(a, Comparator.comparingInt(o -> o[2]));
```

```
    int x = a[0][2];
```

```
    ans.add(a[0][0]);
```

```
    for (int i = 1; i < a.length; i++)
```

```
        if (a[i][1] > x) {
```

```
            ans.add(a[i][0]);
```

```
            x = a[i][2];
```

```
    }
```

```
    Collections.sort(ans);
```

```
    return ans;
```

Wine Selling

```
int wineSelling(int a[], int n) {  
    int b = 0, s = 0;  
    int ans = 0;
```

```
    while (b < n && s < n) {  
        while (a[b] <= 0) {  
            b++;  
        }  
        if (b == n) return ans;
```

```
        while (a[s] >= 0) {  
            s++;  
        }  
        if (s == n)  
            return ans;
```

```
        if (Math.abs(a[b]) >= Math.abs(a[s])) {
```

```
            ans += Math.abs(b - s) * abs(a[s]);  
            a[b] += a[s];
```

```
        }  
        else {
```

```
            ans += abs(b - s) * a[b];  
            a[s] += a[b];  
            a[b] = 0;
```

```
        }  
    }  
    return ans;
```

```
}
```

? Minimum platform

static int findPlatform(int a[], int d[], int n)

Arrays.sort(a);
sort(d);

int count = 0, ans = 0, i = 0, j = 0;

while (i < n) {

if (a[i] <= d[j]) {

count++;

ans = max(ans, count);

i++;

else if (a[i] > d[j]) {

count--;

j++;

}

return ans;

}

TRIE

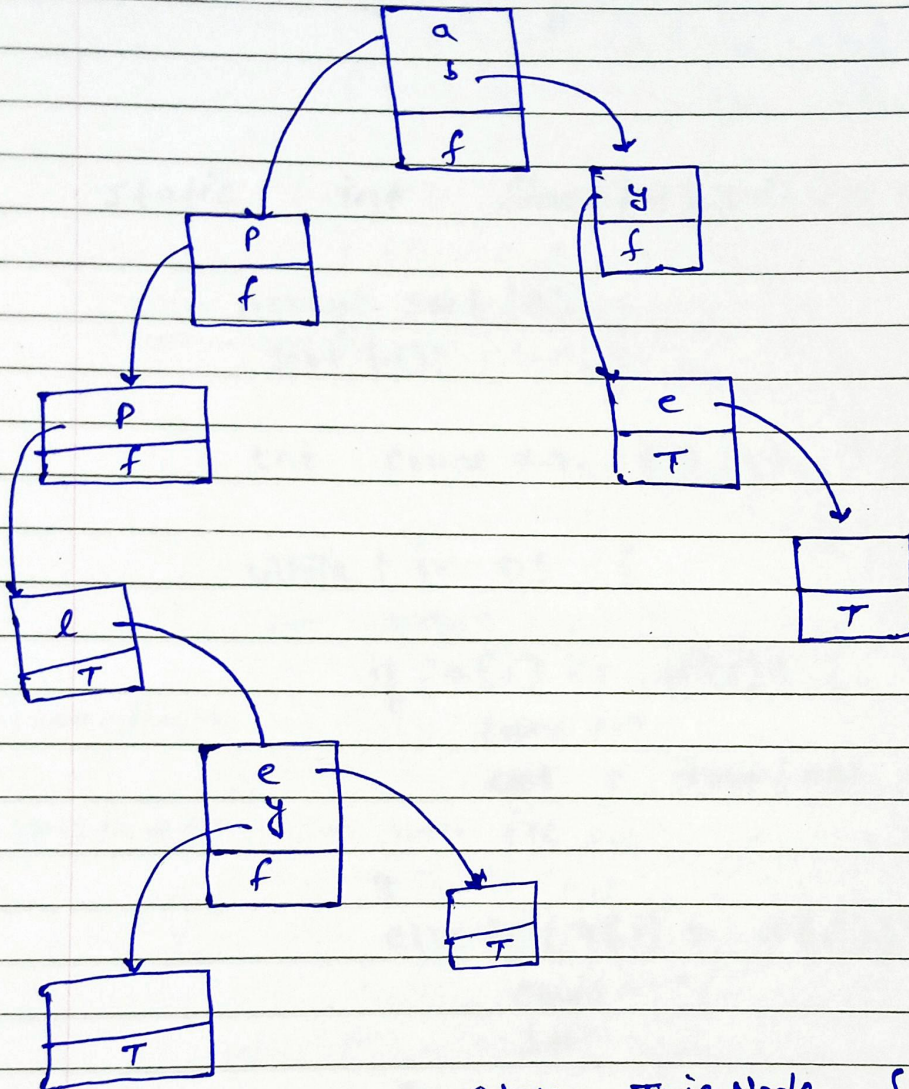
'app'

bye

, apple

by

apply



```

class TrieNode {
    TrieNode [] children;
    boolean isEndOfWord;
}
    
```

constructor {

```

children = new TrieNode[26];
isEndOfWord = false;
    
```

Binary Search.

11 1st and last occurrence of an element.

```
Public int[] searchRange(int[] nums, int target) {  
    int ans[] = {-1, -1};  
    ans[0] = binarySearch(nums, target, true);  
    if (ans[0] != -1)  
        ans[1] = binarySearch(nums, target, false);  
    return ans;  
}
```

3

```
Public int binarySearch(int nums[], int target, boolean firstStartIndex)  
int start = 0, middle, end = nums.length - 1; ans = -1;
```

```
while (start <= end) {  
    middle = start +  $\frac{(end - start)}{2}$ ;
```

```
    if (target > nums[middle])
```

```
        start = middle + 1;
```

```
    else if (target < nums[middle])
```

```
        end = middle - 1;
```

```
    else {
```

```
        ans = middle;
```

```
        if (firstStartIndex)
```

```
            end = middle - 1;
```

```
        else
```

```
            start = middle + 1;
```

3

```
    return ans;  
}
```

3

11 Count of an Element in a Sorted Array.

11 Number of times a Sorted array is Rotated.

```
int findKRotation(int arr[], int n) {
```

```
    int l = 0, r = n - 1;
```

```
    while (l < r) {
```

```
        int m = l + (r - l) / 2;
```

```
        if (arr[m] == arr[r]) r--;
```

```
        else if (arr[m] > arr[r])
```

```
            l = m + 1;
```

```
        else
```

```
            r = m;
```

```
    }
```

```
    return r;
```

```
}
```

11 Search an element in Rotated Sorted Array.

```
public int search(int a[], int target) {
```

```
    int l = 0, h = a.length - 1;
```

```
    while (l <= h) {
```

```
        int m = (l + h) / 2;
```

```
        if (a[m] == target) return m;
```

```
        else if (a[l] <= a[m]) {
```

```
            if (a[l] <= target && a[m] > target) h = m - 1;
```

```
            else l = m + 1;
```

```
        } else {
```

```
            if (a[m] <= target && a[h] > target) l = m + 1;
```

```
            else h = m - 1;
```

```
        }
```

```
    } return -1;
```

```
}
```

11 Search an element in an almost Sorted Array.

```
int binarySearch(int arr[], int l, int r, int x) {
```

```
    if (r >= l) {
```

```
        int mid = l + (r - l) / 2;
```

```
        if (arr[mid] == x)
```

```
            return mid;
```

```
        if (mid > l && arr[mid - 1] == x)
```

```
            return (mid - 1);
```

```
        if (mid < r && arr[mid + 1] == x)
```

```
            return (mid + 1);
```

```
        if (arr[mid] > x)
```

```
            return binarySearch(arr, l, mid - 2, x);
```

```
        return binarySearch(arr, mid + 2, r, x);
```

```
    }  
    return -1;
```

3

// ceil of an element in a sorted array.

// find floor of an element in a sorted array.

```
static int findFloor(long arr[], int n, long x) {  
    return floor(arr, 0, n-1, x);  
}
```

3

```
static int floor(long arr[], int l, int r, long k) {  
    if (l > r) return -1;  
    int mid = (l+r)/2;  
    if (arr[mid] == k)  
        return mid;  
    else if (arr[mid] < k)  
        return Math.max(mid, floor(arr, mid+1, r, k));  
    else  
        return floor(arr, l, mid-1, k);  
}
```

3

// Next Alphabetical Element

```
public char nextGreatestLetter(char letters[], char target) {
```

```
    char res = letters[0];
```

```
    int start = 0, end = letters.length - 1;
```

```
    while (start <= end) {
```

```
        int mid = start + (end - start) / 2;
```

```
        if (letters[mid] == target)
```

```
            start = mid + 1;
```

```
        else if (target > letters[mid])
```

```
            start = mid + 1;
```

```
        else if (letters[mid] > target) {
```

```
            res = letters[mid];
```

```
            end = mid - 1;
```

```
    }
```

```
    return res;
```

```
}
```

// Find the position of an element in an infinite Sorted Array.

```
static int binarySearch(int arr[], int l, int r, int x) {
```

```
    if (l >= r) {
```

```
        int mid = l + (r - l) / 2;
```

```
        if (arr[mid] > x)
```

```
            return binarySearch(arr, l, mid - 1, x);
```

```
        if (arr[mid] == x)
```

```
            return mid;
```

```
        return binarySearch(arr, mid + 1, r, x); } return -1;
```

Static int findPos(int arr[], int key) {

int l = 0, h = 1, val = arr[0];

while(val < key) {

l = h;

if(2*h < arr.length-1)

h = 2*h;

else

h = arr.length-1;

val = arr[h];

}

return binarySearch(arr, l, h, key);

3

Suppose element
found at 1600 index
then number of
operation/comparison
we have to do
is $\log 1600$

$$2^{10} = 1024 \rightarrow 1024$$

$$2^{11} = 2048 \rightarrow 1024$$

// find the index of first 1 in an infinite sorted array of 0s and 1s.

```
public static int indexOfFirstOne(int arr[], int low, int high) {
```

```
    int mid = 0;
```

```
    while (low <= high) {
```

```
        mid = (low + high) / 2;
```

```
        if (arr[mid] == 1 && (mid == 0 || arr[mid-1] == 0))  
            break;
```

```
        else if (arr[mid] == 1)
```

```
            high = mid - 1;
```

```
        else
```

```
            low = mid + 1;
```

```
    }
```

```
    return mid;
```

```
}
```

```
public static int posOfFirstOne(int arr[]) {
```

```
    int l = 0, h = 1;
```

```
    while (arr[h] == 0) {
```

```
        l = h;
```

```
        h = 2 * h;
```

```
    }
```

```
    return indexOfFirstOne(arr, l, h);
```

```
}
```

11 Find maximum element in Bitonic Array.

11 Find Peak Element

Method - 1

```
public int findPeakElement(int nums[]) {  
    return helper(num, 0, num.length - 1);  
}
```

3

```
public int helper(int num[], int start, int end) {  
    if (start == end)  
        return start;  
    else if (start + 1 == end) {  
        if (num[start] > num[end])  
            return start;  
        return end;  
    }
```

3

```
    else {
```

```
        int m = (start + end) / 2;
```

```
        if (num[m] > num[m-1] && num[m] > num[m+1])  
            return m;
```

```
        else if (num[m-1] > num[m] && num[m] > num[m+1])  
            return helper(num, start, m-1);
```

```
        else
```

```
            return helper(num, m+1, end);  
    }
```

3

3

m-2

```
public int findPeakElement(int[] a) {
```

```
    int low = 0, mid = 0, high = a.length - 1;
```

```
    while (low < high) { mid = (low + high) / 2;
```

```
        if (a[mid] < a[mid + 1]) low = mid + 1;
```

```
        else high = mid;
```

```
    }  
    return low;
```

2

// minimum Difference Element in a Sorted Array -

```
while (low <= high) {  
    int mid = low + (high - low) / 2;
```

```
    if (key == arr[mid])
```

```
        return arr[mid];
```

```
    else if (key < arr[mid])
```

```
        high = mid - 1;
```

```
    else
```

```
        low = mid + 1;
```

3

```
min (abs(arr[low] - key), abs(arr[high] - key)).
```

// Allocate Minimum Number of pages.

```
Public int books(ArrayList<Integer> A, int B) {  
    return findPages(A, A.size(), B);  
}
```

3

```
Public int findPages(ArrayList<Integer> A, int totalStudents,  
    int allowedStudents) {
```

```
    long Sum = 0;
```

```
    if (totalStudents < allowedStudents)
```

```
        return -1;
```

```
    for (int i = 0; i < totalStudents; i++)
```

```
        Sum += A.get(i);
```

```
    int start = 0, end = (int)Sum, res = MAX_VALUE;
```

```
    while (start <= end) {
```

```
        int mid = (start + end) / 2;
```

```
        if (isPossible(A, totalStudents, allowedStudents, mid)) {
```

```
            res = Math.min(res, mid);
```

```
            end = mid - 1;
```

```
        }
```

```
        else
```

```
            start = mid + 1;
```

```
    }
```

```
    return res;
```

3

```
Public boolean isPossible(ArrayList<Integer> A, int totalStu-  
    dents, int allowedStudents, int currMin) {  
    int studentsReq = 1, currSum = 0;
```

Recursion में Auxiliary Space use करते हैं
वर्ना iteration में कोई किसेसा Space use नहीं होता है।

```
for(int i=0; i<totalStudents; i++) {  
    if(A.get(i) > currMin)  
        return false;
```

```
    if(currSum + A.get(i) > currMin) {  
        studentsReq++;  
        currSum = A.get(i);  
        if(studentsReq > allowedStudents)  
            return false;
```

```
    }
```

```
    else
```

```
        currSum += A.get(i);
```

```
    }
```

```
    return true;
```

```
}
```

that's why iteration is
better than Recursion

// Search in 2D Matrix (Sorted row wise and column wise)

```
public boolean SearchMatrix(int matrix[][], int target) {
```

```
    if (matrix.length == 0)
```

```
        return false;
```

```
    int rows = matrix.length, columns = matrix[0].length;
```

```
    int low = 0, high = (rows * columns - 1);
```

```
    while (low <= high) {
```

```
        int mid = (low + (high - low) / 2);
```

```
        int value = matrix[mid / columns][mid % columns];
```

```
        if (value == target)
```

```
            return true;
```

```
        else if (value < target)
```

```
            low = mid + 1;
```

```
        else
```

```
            high = mid - 1;
```

```
    }
```

```
    return false;
```

```
}
```

// or.

```
public boolean SearchMatrix(int a[][], int target) {
```

```
    int i = 0;
```

```
    int j = a[0].length - 1;
```

```
    while (i < a.length && i >= 0 && j < a[0].length && j >= 0)
```

```
    {
```

```
        if (a[i][j] == target) return true;
```

```
        else if (a[i][j] > target) j--;
```

```
        else if (a[i][j] < target) i++;
```

```
    }
```

```
    return false;
```

```
}
```